

实验报告

张晨轩 2024K8009929011

1 互联网协议分析实验

1.1 实验内容

本实验用mininet搭建网络环境，在节点h1上启动wireshark，对 h1-eth0 网卡抓包，再用 `wget` 获取 `www.ucas.ac.cn` 的页面。

结合wireshark捕获的数据包，查看ARP、DNS、TCP、HTTP/HTTPS等协议在网页获取过程中起到的作用，以及不同协议之间的封装关系，了解h1获取页面时各协议是怎样运行的。

1.2 实验流程

- 启动带有NAT功能的mininet网络，命令为 `sudo mn --nat`
- 在mininet中打开h1的终端，输入 `mininet> xterm h1`
- 在h1中配置DNS服务器，使用 `echo "nameserver 8.8.8.8" > /etc/resolv.conf`
- 在h1中输入 `wireshark &`，启动wireshark
- 在wireshark中选中 h1-eth0 网卡，开始抓包
- 用以下命令访问 UCAS 网站：`wget -U "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/123.0 Safari/537.36" https://www.ucas.ac.cn`

网页访问结束后，停止wireshark抓包，再分别查看ARP、DNS、TCP和HTTPS数据包并进行分析。

1.3 实验结果及分析

1.3.1 ARP协议分析

在wireshark中把过滤条件设为 `arp`

过滤后能看到h1发出的ARP请求：

```
1 | who has 10.0.0.3? Tell 10.0.0.1
```

随后收到了与请求对应的ARP响应：

```
1 | 10.0.0.3 is at ce:73:a6:35:88:a5
```

抓包结果显示，h1的IP地址为 10.0.0.1，需要把数据发送到下一跳 10.0.0.3；数据链路层发送 Ethernet帧时，需要知道下一跳设备的MAC地址，所以h1先用ARP查询 10.0.0.3 对应的MAC地址，收到的ARP响应给出了该地址对应的MAC地址：

```
1 | ce:73:a6:35:88:a5
```

在本次实验中，ARP由此完成了下一跳IP地址到MAC地址的映射。

ucas_ac_cn.pcapng						
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help						
arp						
No.	Time	Source	Destination	Protocol	Length	Info
3	18.797812233	ae:fa:e2:66:a3:ba	Broadcast	ARP	42	Who has 10.0.0.3? Tell 10.0.0.1
4	18.798078533	ce:73:a6:35:88:a5	ae:fa:e2:66:a3:ba	ARP	42	10.0.0.3 is at ce:73:a6:35:88:a5
45	29.207955905	ce:73:a6:35:88:a5	ae:fa:e2:66:a3:ba	ARP	42	Who has 10.0.0.1? Tell 10.0.0.3
46	29.207965571	ae:fa:e2:66:a3:ba	ce:73:a6:35:88:a5	ARP	42	10.0.0.1 is at ae:fa:e2:66:a3:ba

1.3.2 DNS协议分析

在wireshark中把 dns 设为过滤条件

过滤结果中能看到h1向DNS服务器 8.8.8.8 发出的查询请求，A记录的查询内容如下：

```

1 Name: www.ucas.ac.cn
2 Type: A
3 Class: IN

```

```

▶ Frame 5: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface h1-eth0, id 0
▶ Ethernet II, Src: ae:fa:e2:66:a3:ba (ae:fa:e2:66:a3:ba), Dst: ce:73:a6:35:88:a5 (ce:73:a6:35:88:a5)
▶ Internet Protocol Version 4, Src: 10.0.0.1, Dst: 8.8.8.8
▶ User Datagram Protocol, Src Port: 39525, Dst Port: 53
▼ Domain Name System (query)
  Transaction ID: 0xcfc6
  ▶ Flags: 0x0100 Standard query
  Questions: 1
  Answer RRs: 0
  Authority RRs: 0
  Additional RRs: 0
  ▼ Queries
    ▼ www.ucas.ac.cn: type A, class IN
      Name: www.ucas.ac.cn
      [Name Length: 14]
      [Label Count: 4]
      Type: A (1) (Host Address)
      Class: IN (0x0001)

```

随后，DNS服务器返回了响应，内容包括：

```

1 Name: www.ucas.ac.cn
2 Type: A
3 Address: 124.16.77.5

```

根据本次实验的响应结果，www.ucas.ac.cn 的IP地址为 124.16.77.5

```

▶ Frame 10: 90 bytes on wire (720 bits), 90 bytes captured (720 bits) on interface h1-eth0, id 0
▶ Ethernet II, Src: ce:73:a6:35:88:a5 (ce:73:a6:35:88:a5), Dst: ae:fa:e2:66:a3:ba (ae:fa:e2:66:a3:ba)
▶ Internet Protocol Version 4, Src: 8.8.8.8, Dst: 10.0.0.1
▶ User Datagram Protocol, Src Port: 53, Dst Port: 39525
▼ Domain Name System (response)
  Transaction ID: 0xcfc6
  ▶ Flags: 0x8180 Standard query response, No error
  Questions: 1
  Answer RRs: 1
  Authority RRs: 0
  Additional RRs: 0
  ▼ Queries
    ▼ www.ucas.ac.cn: type A, class IN
      Name: www.ucas.ac.cn
      [Name Length: 14]
      [Label Count: 4]
      Type: A (1) (Host Address)
      Class: IN (0x0001)
  ▼ Answers
    ▼ www.ucas.ac.cn: type A, class IN, addr 124.16.77.5
      Name: www.ucas.ac.cn
      Type: A (1) (Host Address)
      Class: IN (0x0001)
      Time to live: 3193 (53 minutes, 13 seconds)
      Data length: 4
      Address: 124.16.77.5
      [Request In: 7]
      [Time: 0.239727117 seconds]

```

该DNS请求的Transaction ID为 0xcfc6，还能看到以下内容：

1	Answer RRs: 1
2	Request In: 7
3	Time: 0.239727117 seconds

由此可知，从对应请求发出到收到响应，经过的时间约为0.24s

抓包时还看到了AAAA类型的DNS查询，A记录用来查询域名对应的IPv4地址，AAAA记录则用来查询IPv6地址。

本实验中，DNS把用户输入的域名 `www.ucas.ac.cn` 映射为IP地址 `124.16.77.5`，供网络层使用。

1.3.3 TCP协议分析

完成DNS解析后，h1开始与服务器 `124.16.77.5` 建立TCP连接。

这次连接的基本信息记录如下：

1	客户端 IP: 10.0.0.1
2	客户端端口: 44642
3	服务器 IP: 124.16.77.5
4	服务器端口: 443
5	TCP Stream index: 0

在Wireshark中能看到TCP三次握手的顺序：`SYN -> SYN,ACK -> ACK`

其中，`10.0.0.1:44642` 发起 `SYN` 和 `ACK`，对 `SYN,ACK` 作出应答；`124.16.77.5:443` 发起 `SYN,ACK`，应答的内容为 `SYN` 和 `ACK`

三次握手各自对应的时间如下：

```
1 | SYN: 24.042223435 s
2 | SYN, ACK: 24.043229145 s
3 | ACK: 24.043261499 s
```

根据记录，从 SYN 到 SYN, ACK 的时间差约为**1.01ms**

11	24.042223435	10.0.0.1	124.16.77.5	TCP	74	44642 → 443	[SYN] Seq=0 Win=
12	24.043229145	124.16.77.5	10.0.0.1	TCP	74	443 → 44642	[SYN, ACK] Seq=0
13	24.043261499	10.0.0.1	124.16.77.5	TCP	66	44642 → 443	[ACK] Seq=1 Ack=

再打开第一个SYN包，能看到以下信息：

```
1 | Source Port: 44642
2 | Destination Port: 443
3 | Sequence Number: 0
4 | Flags: SYN
```

```
▶ Frame 11: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface h1-eth0, id 0
▶ Ethernet II, Src: ae:fa:e2:66:a3:ba (ae:fa:e2:66:a3:ba), Dst: ce:73:a6:35:88:a5 (ce:73:a6:35:88:a5)
▶ Internet Protocol Version 4, Src: 10.0.0.1, Dst: 124.16.77.5
▼ Transmission Control Protocol, Src Port: 44642, Dst Port: 443, Seq: 0, Len: 0
  Source Port: 44642
  Destination Port: 443
  [Stream index: 0]
  ▶ [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 3837630873
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  1010 ... = Header Length: 40 bytes (10)
  ▶ Flags: 0x002 (SYN)
  Window: 42340
  [Calculated window size: 42340]
  Checksum: 0xd344 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  ▶ Options: (20 bytes), Maximum segment size, SACK permitted, Timestamps, No-Operation (NOP), Window scale
  ▶ [Timestamps]
```

TCP 三次握手时，客户端先发送SYN来请求建立连接，服务器返回SYN和 ACK，表示同意建立连接，并确认已收到客户端的请求；客户端再发送 ACK 作出确认，三次握手完成后，双方的TCP连接便建立了。

1.3.4 HTTPS协议分析

本实验访问的地址是 `https://www.ucas.ac.cn`，服务器端口为 `443`，所以建立TCP连接后，还要进行TLS握手。

在wireshark中能看到TLS Client Hello数据包，包含以下内容：

```
1 | Handshake Protocol: Client Hello
2 | Server Name: www.ucas.ac.cn
```

```
Transport Layer Security
  TLSv1.2 Record Layer: Handshake Protocol: Client Hello
    Content Type: Handshake (22)
    Version: TLS 1.0 (0x0301)
    Length: 493
    Handshake Protocol: Client Hello
      Handshake Type: Client Hello (1)
      Length: 399
      Versions: TLS 1.2 (0x0303)
      Random: c978530785eb2631c8d5dd0ce06861069eeac2163140ebdefa22ee4086a0769
      Session ID Length: 32
      Session ID: 17a8343b1982dab0c2e9f92a79d939452b81e566e1cd427f61632e18749b203a
      Cipher Suites Length: 150
      Compression Methods Length: 1
      Compression Methods (1 method)
      Extensions Length: 176
      Extension: server_name (len=19) name=www.ucas.ac.cn
      Extension: ec_point_formats (len=4)
      Extension: supported_groups (len=22)
      Extension: session_ticket (len=0)
      Extension: encrypt_then_mac (len=0)
      Extension: extended_master_secret (len=0)
      Extension: post_handshake_auth (len=0)
      Extension: signature_algorithms (len=42)
      Extension: supported_versions (len=5) TLS 1.3, TLS 1.2
      Extension: psk_key_exchange_modes (len=2)
      Extension: key_share (len=38) x25519
      [JA4: t13d751100.479067518aa3.f08d57fd48c1]
      [JA4_r (truncated): t13d751100.062f.0632.0633.0635.0638.0639.063c.063d.0640.0641.0644.0645.0667.066a.066b.0684.0687.0688.069c.069d.069e.069f.06a2.06a3.06ba.06bd.06be.06c0.06c3.06c4.06ff.1301.1302.1303.c009.c00a.c013.c014.c023.c024.c027.c028]
      [JA3 Fullstring (truncated): 771.4866-4867-4865-49196-49200-163-159-52393-52392-52394-49327-49325-49315-49311-49245-49249-49239-49235-49195-49199-162-158-49326-49324-49314-49310-49244-49248-49236-49234-49188-49192-167-166-49267-49271-196-19]
      [JA3: e0ce64088cf382a3a56b713620f0294]
```

HTTPS在HTTP与TCP之间加入了TLS加密层，建立TCP连接后，还要通过TLS握手协商后续加密通信需要的信息；完成这一步后，HTTP请求和服务器返回的数据会通过TLS加密传输。

因此，普通wireshark抓包无法直接看到完整的HTTP明文请求内容，本次把过滤器设为 http 后，没有找到符合条件的包。

1.3.5 协议封装关系

查看DNS查询数据包，能看到：

1	Ethernet II
2	Internet Protocol Version 4
3	User Datagram Protocol
4	Domain Name System

对应的协议封装关系为 Ethernet < IP < UDP < DNS

No.	Time	Source	Destination	Protocol	Length	Info
5	18.798081928	10.0.0.1	8.8.8.8	DNS	74	Standard query 0xcfc6 A www.ucas.ac.cn
6	18.798082337	10.0.0.1	8.8.8.8	DNS	74	Standard query 0x8dc7 AAAA www.ucas.ac.cn
7	23.802340041	10.0.0.1	8.8.8.8	DNS	74	Standard query 0xcfc6 A www.ucas.ac.cn
8	23.802370161	10.0.0.1	8.8.8.8	DNS	74	Standard query 0x8dc7 AAAA www.ucas.ac.cn
9	24.042066270	8.8.8.8	10.0.0.1	DNS	132	Standard query response 0x8dc7 AAAA www.ucas.ac.cn
10	24.042067158	8.8.8.8	10.0.0.1	DNS	90	Standard query response 0xcfc6 A www.ucas.ac.cn

Frame 7: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface eth0	0000	ce 73 a6 35 88 a5 ae fa e2 66 a3 ba 08 00 45 00
Ethernet II, Src: ae:fa:e2:66:a3:ba (ae:fa:e2:66:a3:ba), Dst: 08:00:00:00:00:00	0010	00 3c 8f 80 40 00 40 11 91 20 0a 00 00 01 08 08
Internet Protocol Version 4, Src: 10.0.0.1, Dst: 8.8.8.8	0020	08 08 9a 65 00 35 00 28 1a 4a cf c6 01 00 00 01
User Datagram Protocol, Src Port: 39525, Dst Port: 53	0030	00 00 00 00 00 00 03 77 77 77 04 75 63 61 73 02
Domain Name System (query)	0040	61 63 02 63 6e 00 00 01 00 01

查看HTTPS数据包，能看到：

1	Ethernet II
2	Internet Protocol Version 4
3	Transmission Control Protocol
4	Transport Layer Security

对应的封装关系为 Ethernet < IP < TCP < TLS

No.	Time	Source	Destination
11	24.042223435	10.0.0.1	124.16.77.5
12	24.043229145	124.16.77.5	10.0.0.1
13	24.043261499	10.0.0.1	124.16.77.5
14	24.055594528	10.0.0.1	124.16.77.5
15	24.056653809	124.16.77.5	10.0.0.1
16	24.067856429	124.16.77.5	10.0.0.1
17	24.067866850	10.0.0.1	124.16.77.5
18	24.069265253	10.0.0.1	124.16.77.5
19	24.071366465	124.16.77.5	10.0.0.1
20	24.071418246	10.0.0.1	124.16.77.5
21	24.081487799	124.16.77.5	10.0.0.1
22	24.081510572	10.0.0.1	124.16.77.5
23	24.082570769	124.16.77.5	10.0.0.1
24	24.082576432	10.0.0.1	124.16.77.5


```

Frame 14: 474 bytes on wire (3792 bits), 474 bytes captured on interface 0
Ethernet II, Src: ae:fa:e2:66:a3:ba (ae:fa:e2:66:a3:ba), Dst: 10.0.0.1
Internet Protocol Version 4, Src: 10.0.0.1, Dst: 124.16.77.5
Transmission Control Protocol, Src Port: 44642, Dst Port: 80
Transport Layer Security

```

1.3.6 UCAS 页面获取过程

- 用户使用 `wget`，请求访问 `https://www.ucas.ac.cn`
- 此时客户端只知道域名，还没有服务器的IP地址，需要先向DNS服务器 `8.8.8.8` 发送DNS查询，DNS 返回 `www.ucas.ac.cn -> 124.16.77.5`
 - 得到服务器IP地址后，客户端还需要通过网络把数据发送出去
- 发送Ethernet帧前，h1 用ARP查询下一跳 `10.0.0.3` 的MAC地址，得到的结果为 `10.0.0.3 -> ce:73:a6:35:88:a5`
- h1通过本地端口 `44642` 与服务器 `124.16.77.5:443` 建立TCP连接，双方按顺序完成 `SYN`、`SYN,ACK`、`ACK` 三次握手
- 建立TCP连接后，因为访问的是HTTPS网站，客户端还会发送TLS Client Hello，与服务器建立加密连接
 - 随后的HTTP请求及网页响应通过TLS加密，后续数据传输由TCP完成

2 传输性能测量实验

2.1 实验内容

本实验在Linux环境下用python编写测量脚本，对约1000个网站的网络传输性能进行测量。

测量主要记录以下几项指标：

- Ping
- DNS
- TCP transfer
- Total time

测量时也记录了TCP握手时间，便于进一步查看传输过程中不同阶段各自的时间开销。

获得网站服务器的 IP 地址后，用GeoIP数据库查询服务器的大致地理位置，再根据客户端与服务器之间的地理距离，估算理论传播时延c-latency，并把实际网络传输性能与 c-latency 作比较。

2.2 实验流程

2.2.1 测试网站选取

从 Tranco 网站排名数据中，按照排名顺序依次读取域名。

部分网站在当前网络环境下不能正常解析或访问，因此编写了筛选脚本，对候选网站逐个做DNS解析和HTTP/HTTPS可访问性测试

筛选脚本共选出了 1000 个能正常解析和访问的网站，把结果保存到 `sites.txt`

```
sites.txt
1  cloudflare.com
2  microsoft.com
3  amazonaws.com
4  apple.com
5  mail.ru
6  dzen.ru
7  linkedin.com
8  office.com
9  googletagmanager.com
10 live.com
11 amazon.com
12 azure.com
13 bing.com
14 appsflyersdk.com
15 fastly.net
16 netflix.com
17 sharepoint.com
18 gandi.net
19 digicert.com
20 cloud.microsoft
```

筛选使用的脚本 `filter_sites.py` 如下：

```
1  import csv
2  import socket
3  import subprocess
4  import time
5
6  INPUT_FILE = "tranco_Q2Q44.csv"
7  OUTPUT_FILE = "sites.txt"
8
9  TARGET_COUNT = 1000
10 SITE_TIMEOUT = 8 # 设置最大测试时长
11 CONNECT_TIMEOUT = 3
12
```

```

13
14 def can_resolve(domain):
15     try:
16         socket.setdefaulttimeout(SITE_TIMEOUT)
17         socket.gethostbyname(domain)
18         return True
19     except Exception:
20         return False
21
22
23 def can_access(domain, remaining_time):
24     if remaining_time <= 0:
25         return False
26
27     schemes = ["https", "http"]
28
29     for scheme in schemes:
30         if remaining_time <= 0:
31             return False
32
33         url = f"{scheme}://{domain}"
34
35         max_time = max(1, int(remaining_time))
36
37         try:
38             result = subprocess.run(
39                 [
40                     "curl",
41                     "--noproxy", "*",
42                     "-L",
43                     "-o", "/dev/null",
44                     "-s",
45                     "--connect-timeout", str(CONNECT_TIMEOUT),
46                     "--max-time", str(max_time),
47                     url,
48                 ],
49                 stdout=subprocess.DEVNULL,
50                 stderr=subprocess.DEVNULL,
51                 timeout=remaining_time + 1,
52             )
53
54             if result.returncode == 0:
55                 return True
56
57         except subprocess.TimeoutExpired:
58             return False
59         except Exception:
60             pass
61
62         return False
63
64     return False
65

```

```

66
67 def valid_site(domain):
68     start = time.monotonic()
69
70     if not can_resolve(domain):
71         return False
72
73     elapsed = time.monotonic() - start
74     remaining = SITE_TIMEOUT - elapsed
75
76     if remaining <= 0:
77         return False
78
79     return can_access(domain, remaining)
80
81
82 def main():
83     count = 0
84     checked = 0
85
86     with open(INPUT_FILE, "r", encoding="utf-8-sig", newline="") as f, \
87         open(OUTPUT_FILE, "w", encoding="utf-8") as out:
88
89         reader = csv.reader(f)
90
91         for row in reader:
92             if len(row) < 2:
93                 continue
94
95             domain = row[1].strip()
96
97             if not domain:
98                 continue
99
100             checked += 1
101
102             print(
103                 f"[checked={checked}, valid={count}/{TARGET_COUNT}] "
104                 f"testing {domain} ..."
105             )
106
107             start = time.monotonic()
108             ok = valid_site(domain)
109             elapsed = time.monotonic() - start
110
111             if ok:
112                 count += 1
113                 out.write(domain + "\n")
114                 out.flush()
115
116             print(
117                 f" OK ({elapsed:.2f}s) "
118                 f"-> saved as #{count}"

```

```

119         )
120     else:
121         print(
122             f"    FAIL ({elapsed:.2f}s)"
123         )
124
125     if count >= TARGET_COUNT:
126         break
127
128     print()
129     print(f"checked sites: {checked}")
130     print(f"valid sites:    {count}")
131     print(f"output:         {OUTPUT_FILE}")
132
133
134 if __name__ == "__main__":
135     main()
136

```

2.2.2 网络性能测量

用python编写 `measure.py`，依次读取 `sites.txt` 中的网站并进行测量。

测量每个网站时，先通过 DNS 解析得到服务器的 IPv4 地址，再用 `ping` 测量网络 RTT；同时用 `curl` 获取 DNS 解析、TCP 建连，以及整个页面传输过程的时间信息。

程序会把以下字段记录下来：

```

1  rank
2  domain
3  ip
4  ping_ms
5  dns_ms
6  tcp_handshake_ms
7  tcp_transfer_ms
8  total_ms
9  url

```

各项时间的计算方式如下：

- `DNS time = time_namelookup`
- TCP三次握手时间按以下方式计算：`TCP handshake = time_connect - time_namelookup`
- TCP transfer的计算方式为 `TCP transfer = time_total - time_connect`
- 总传输时间记为 `Total time = time_total`

把测量得到的结果保存到 `results.csv`

```

results.csv > data
1 rank,domain,ip,ping_ms,dns_ms,tcp_handshake_ms,tcp_transfer_ms,total_ms,url
2 1,cloudflare.com,104.16.133.229,216.476,17.855,393.04699999999997,11305.25,11716.152,https://cloudflare.com
3 2,microsoft.com,150.171.110.70,,27.743,246.51000000000005,3372.335,3646.5879999999997,https://microsoft.com
4 3,amazonaws.com,15.197.250.250,78.459,20.159,278.141,5922.28,6220.58,http://amazonaws.com
5 4,apple.com,17.253.144.10,98.987,30.862000000000002,67.826,718.8149999999999,817.5029999999999,https://apple.com
6 5,mail.ru,185.180.201.1,235.765,560.838,597.4509999999999,1429.4800000000002,2587.7690000000002,https://mail.ru
7 6,dzen.ru,95.163.218.220,,1044.096,389.62600000000003,1033.9930000000002,2467.715,https://dzen.ru
8 7,linkedin.com,52.131.37.152,11.449,56.501999999999995,34.739000000000004,2699.185,2790.426,https://linkedin.com
9 8,office.com,13.107.6.156,73.008,50.33,204.784,1653.3200000000002,1998.434,https://office.com
10 9,googletagmanager.com,220. Col 4: ping_ms 2,1027.811,17.25399999999998,1137.2839999999999,2182.3489999999997,https://
11 10,live.com,204.79.197.212,87.529,34.389,224.976,1404.262,1663.627,https://live.com
12 11,amazon.com,98.87.170.71,,17.058,331.94399999999996,881.837,1230.839,https://amazon.com
13 12,azure.com,20.76.201.171,287.052,1433.192,284.22299999999999,5678.71,7396.125,https://azure.com
14 13,bing.com,150.171.27.10,84.707,38.3,499.339,956.5920000000001,1494.231,https://bing.com
15 14,appsflyersdk.com,65.9.168.12,93.896,56.583,101.77,179.162,337.515,https://appsflyersdk.com
16 15,fastly.net,151.101.65.6,185.543,349.967,264.98800000000006,8213.378999999999,8828.334,https://fastly.net
17 16,netflix.com,44.242.13.161,,21.298000000000002,184.60399999999998,426.3699999999999,632.2719999999999,https://netfli
18 17,sharepoint.com,13.107.136.10,59.285,151.881,717.157,2803.394,3672.4320000000002,https://sharepoint.com
19 18,gandi.net,217.70.185.65,177.623,336.911,340.08500000000004,1637.396,2314.392,https://gandi.net
20 19,digicert.com,45.60.121.229,152.995,148.208,480.033,1287.629,1915.87,https://digicert.com
21 20,cloud.microsoft,20.76.201.171,252.945,38.031,251.15300000000002,577.851,867.035,https://cloud.microsoft
22 21,wordpress.org,66.6.42.252,229.183,19.227,274.132,1560.315,1853.674,https://wordpress.org
23 22,skype.com,20.112.250.133,139.181,38.382,376.6,1636.48,2051.462,https://skype.com
24 23,icloud.com,17.253.144.10,84.73,28.014,184.28399999999996,550.923,763.221,https://icloud.com
25 24,cloudflare.net,104.16.208.90,186.487,15.952000000000002,163.072,341.18500000000006,520.2090000000001,https://cloudf
26 25,roblox.com,128.116.31.3,233.632,186.96099999999998,483.40299999999996,1263.973,1934.337,https://roblox.com
27 26,yahoo.com,98.137.11.164,189.018,28.055,387.545,782.173,1197.773,https://yahoo.com
28 27,ntp.org,204.93.207.22,271.378,18.227,399.553,1431.1829999999998,1848.963,https://ntp.org
29 28,msn.com,204.79.197.219,89.496,38.119,463.8639999999999,1339.169,1841.1519999999998,https://msn.com
30 29,adobe.com,23.2.16.113,274.597,20.657999999999998,444.29799999999994,5238.044000000001,5703.0,https://adobe.com

```

本次一共测试了 1000 个网站，有效结果统计如下：

- 1 测试网站总数：1000
- 2 IP 解析成功：1000
- 3 Ping 测量成功：824
- 4 HTTP 测量成功：992

测量脚本 `measure.py` 如下：

```

1 import csv
2 import re
3 import socket
4 import subprocess
5 import time
6
7 INPUT_FILE = "sites.txt"
8 OUTPUT_FILE = "results.csv"
9
10 PING_COUNT = 3
11 TIMEOUT = 10
12
13
14 def resolve_ip(domain):
15     try:
16         return socket.gethostbyname(domain)
17     except Exception:
18         return ""
19
20
21 def measure_ping(domain):

```

```

22     try:
23         result = subprocess.run(
24             ["ping", "-c", str(PING_COUNT), "-w", "2", domain],
25             stdout=subprocess.PIPE,
26             stderr=subprocess.DEVNULL,
27             text=True,
28             timeout=10,
29         )
30
31         m = re.search(
32             r"rtt min/avg/max/mdev = [\d.]+/([\d.]+)/",
33             result.stdout
34         )
35
36         if m:
37             return float(m.group(1))
38
39     except Exception:
40         pass
41
42     return None
43
44
45 def measure_curl(domain):
46     fmt = (
47         "%{time_name}lookup},"
48         "%{time_name}connect},"
49         "%{time_name}total}"
50     )
51
52     urls = [
53         f"https://{domain}",
54         f"http://{domain}",
55     ]
56
57     for url in urls:
58         try:
59             result = subprocess.run(
60                 [
61                     "curl",
62                     "--noproxy", "*",
63                     "-L",
64                     "-o", "/dev/null",
65                     "-s",
66                     "--connect-timeout", str(TIMEOUT),
67                     "--max-time", "20",
68                     "-w", fmt,
69                     url,
70                 ],
71                 stdout=subprocess.PIPE,
72                 stderr=subprocess.DEVNULL,
73                 text=True,
74                 timeout=25,

```

```

75         )
76
77         if result.returncode != 0:
78             continue
79
80         values = result.stdout.strip().split(",")
81
82         if len(values) != 3:
83             continue
84
85         dns = float(values[0])
86         connect = float(values[1])
87         total = float(values[2])
88
89         if total <= 0:
90             continue
91
92         dns_ms = dns * 1000
93         tcp_handshake_ms = max(0, (connect - dns) * 1000)
94         tcp_transfer_ms = max(0, (total - connect) * 1000)
95         total_ms = total * 1000
96
97         return (
98             dns_ms,
99             tcp_handshake_ms,
100            tcp_transfer_ms,
101            total_ms,
102            url,
103        )
104
105     except Exception:
106         continue
107
108     return None
109
110
111 def main():
112     with open(INPUT_FILE, "r", encoding="utf-8") as f:
113         domains = [
114             line.strip()
115             for line in f
116             if line.strip()
117         ]
118
119     with open(
120         OUTPUT_FILE,
121         "w",
122         newline="",
123         encoding="utf-8"
124     ) as f:
125
126         writer = csv.writer(f)
127

```

```

128     writer.writerow([
129         "rank",
130         "domain",
131         "ip",
132         "ping_ms",
133         "dns_ms",
134         "tcp_handshake_ms",
135         "tcp_transfer_ms",
136         "total_ms",
137         "url",
138     ])
139
140     for rank, domain in enumerate(domains, 1):
141         print(f"[{rank}/{len(domains)}] {domain}")
142
143         ip = resolve_ip(domain)
144         ping_ms = measure_ping(domain)
145         curl_result = measure_curl(domain)
146
147         if curl_result:
148             (
149                 dns_ms,
150                 handshake_ms,
151                 transfer_ms,
152                 total_ms,
153                 url,
154             ) = curl_result
155         else:
156             dns_ms = None
157             handshake_ms = None
158             transfer_ms = None
159             total_ms = None
160             url = ""
161
162         writer.writerow([
163             rank,
164             domain,
165             ip,
166             ping_ms,
167             dns_ms,
168             handshake_ms,
169             transfer_ms,
170             total_ms,
171             url,
172         ])
173
174         f.flush()
175
176         time.sleep(0.1)
177
178
179 if __name__ == "__main__":
180     main()

```

2.2.3 GeoIP获取和c-latency计算

2.2.3.1 服务器位置获取

根据 `results.csv` 中得到的服务器 IP 地址，在DB-IP City Lite GeoIP数据库中查询对应的大致地理位置，记录的内容包括：

```
1 country
2 city
3 latitude
4 longitude
```

客户端位置以实验所在的北京市怀柔区作近似，经纬度按以下数值设置：

```
1 Latitude = 40.3160
2 Longitude = 116.6318
```

python脚本按上述设置计算距离，再把结果保存到 `results_geo.csv`

```
results_geo.csv > data
1 rank, domain, ip, ping_ms, dns_ms, tcp_handshake_ms, tcp_transfer_ms, total_ms, url, server_lat, server_lon, country, city, distance
2 1, cloudflare.com, 104.16.133.229, 216.476, 17.855, 393.047, 11305.25, 11716.152, https://cloudflare.com, 43.6532, -79.3832, Canada, Montreal, 11716.152
3 2, microsoft.com, 150.171.110.70, 27.743, 246.51000000000005, 3372.335, 3646.588, https://microsoft.com, 32.7767, -96.797, United States, Dallas, 32.7767
4 3, amazonaws.com, 15.197.250.250, 78.459, 20.159, 278.141, 5922.28, 6220.58, http://amazonaws.com, 45.5019, -73.5674, Canada, Montreal, 45.5019
5 4, apple.com, 17.253.144.10, 98.987, 30.862, 67.826, 718.8149999999999, 817.5029999999999, https://apple.com, 37.3322, -122.011, United States, Cupertino, 37.3322
6 5, mail.ru, 185.180.201.1, 235.765, 560.838, 597.4509999999999, 1429.4800000000002, 2587.769, https://mail.ru, 55.7951, 37.5418, Russia, Moscow, 55.7951
7 6, dzen.ru, 95.163.218.220, 1044.096, 389.626, 1033.9930000000002, 2467.715, https://dzen.ru, 55.7558, 37.6173, Russia, Moscow, 55.7558
8 7, linkedin.com, 52.131.37.152, 11.449, 56.502, 34.739000000000004, 2699.185, 2790.426, https://linkedin.com, 39.9042, 116.407, United States, New York, 39.9042
9 8, office.com, 13.107.6.156, 73.008, 50.33, 294.784, 1653.3200000000002, 1998.434, https://office.com, 40.7357, -74.1724, United States, New York, 40.7357
10 9, googletagmanager.com, 220.181.174.169, 31.62, 1027.811, 17.253999999999988, 1137.284, 2182.349, https://googletagmanager.com, 40.7128, -74.006, United States, New York, 40.7128
11 10, live.com, 204.79.197.212, 87.529, 34.389, 224.976, 1404.262, 1663.627, https://live.com, 40.8618, -74.0587, United States, New York, 40.8618
12 11, amazon.com, 98.87.170.71, 17.058, 331.944, 881.837, 1230.839, https://amazon.com, 39.0438, -77.4874, United States, Ashburn, 39.0438
13 12, azure.com, 20.76.201.171, 287.052, 1433.192, 284.22299999999999, 5678.71, 7396.125, https://azure.com, 52.3676, 4.90414, The Netherlands, Amsterdam, 52.3676
14 13, bing.com, 150.171.27.10, 84.707, 38.3, 499.339, 956.592, 1494.231, https://bing.com, 40.7128, -74.006, United States, New York, 40.7128
15 14, appsflyersdk.com, 65.9.168.12, 93.896, 56.583, 101.77, 179.162, 337.515, https://appsflyersdk.com, 1.35208, 103.82, Singapore, Singapore, 1.35208
16 15, fastly.net, 151.101.65.6, 185.543, 349.967, 264.98800000000006, 8213.378999999999, 8828.334, https://fastly.net, 45.5019, -73.5674, Canada, Montreal, 45.5019
17 16, netflix.com, 44.242.13.161, 21.298, 184.604, 426.36999999999999, 632.2719999999999, https://netflix.com, 45.5152, -122.678, United States, Los Angeles, 45.5152
18 17, sharepoint.com, 13.107.136.10, 59.285, 151.881, 717.157, 2803.394, 3672.432, https://sharepoint.com, 40.7128, -74.006, United States, New York, 40.7128
19 18, gandi.net, 217.70.185.65, 177.623, 336.911, 340.08500000000004, 1637.396, 2314.392, https://gandi.net, 48.8219, 2.3709, France, Paris, 48.8219
20 19, digicert.com, 45.60.121.229, 152.995, 148.208, 480.033, 1287.629, 1915.87, https://digicert.com, 40.7128, -74.006, United States, New York, 40.7128
21 20, cloud.microsoft.com, 20.76.201.171, 252.945, 38.031, 251.153, 577.851, 867.035, https://cloud.microsoft.com, 52.3676, 4.90414, The Netherlands, Amsterdam, 52.3676
22 21, wordpress.org, 66.6.42.252, 229.183, 19.227, 274.132, 1560.315, 1853.674, https://wordpress.org, 39.0438, -77.4874, United States, Ashburn, 39.0438
23 22, skype.com, 20.112.250.133, 139.181, 38.382, 376.6, 1636.48, 2051.462, https://skype.com, 41.5868, -93.625, United States, Des Moines, 41.5868
24 23, icloud.com, 17.253.144.10, 84.73, 28.014, 184.284, 550.923, 763.221, https://icloud.com, 37.3322, -122.011, United States, Cupertino, 37.3322
25 24, cloudflare.net, 104.16.208.90, 186.487, 15.952000000000002, 163.072, 341.18500000000006, 520.20900000000001, https://cloudflare.com, 43.6532, -79.3832, Canada, Montreal, 43.6532
26 25, roblox.com, 128.116.31.3, 233.632, 186.961, 483.403, 1263.973, 1934.337, https://roblox.com, 51.5072, -0.127586, United Kingdom, London, 51.5072
27 26, yahoo.com, 98.137.11.164, 189.018, 28.055, 387.545, 782.173, 1197.773, https://yahoo.com, 40.7309, -73.9918, United States, New York, 40.7309
28 27, ntp.org, 204.93.207.22, 271.378, 18.227, 399.553, 1431.1829999999998, 1848.963, https://ntp.org, 41.8781, -87.6298, United States, Des Moines, 41.8781
29 28, msn.com, 204.79.197.219, 89.496, 38.119, 463.8639999999999, 1339.169, 1841.152, https://msn.com, 40.8618, -74.0587, United States, New York, 40.8618
30 29, adobe.com, 23.2.16.113, 274.597, 20.658, 444.298, 5238.044000000001, 5703.0, https://adobe.com, 22.3186, 114.18, Hong Kong, Hong Kong, 22.3186
31 30, nginx.org, 3.125.197.172, 167.412, 10.792, 190.449, 564.364, 765.605, https://nginx.org, 50.1109, 8.68213, Germany, Frankfurt, 50.1109
32 31, zoom.us, 170.114.52.2, 180.118, 19.947, 377.538, 1285.381, 1682.866, https://zoom.us, 37.3329, -121.895, United States, San Jose, 37.3329
33 32, qq.com, 123.150.76.218, 42.936, 45.319, 20.812, 65.391, 131.522, https://qq.com, 39.0851, 117.199, China, Youyilu, 39.0851
34 33, baidu.com, 110.242.74.102, 15.766, 42.684, 18.553, 73.254, 134.491, https://baidu.com, 40.9515, 117.963, China, Chengde, 40.9515
35 34, openai.com, 104.18.33.45, 188.452, 16.527, 213.479, 465.795, 695.801, https://openai.com, 43.6532, -79.3832, Canada, Toronto, 43.6532
36 35, yandex.net, 213.180.204.242, 257.963, 15.278, 486.365, 1303.5290000000002, 1805.172, https://yandex.net, 55.7342, 37.5859, Russia, Moscow, 55.7342
37 36, windows.com, 20.70.246.20, 196.02, 128.539, 757.085, 3879.967, 4765.590999999999, https://windows.com, -33.8688, 151.209, Australia, Sydney, -33.8688
```

2.2.3.2 c-latency计算

课件给出的光速网络传播时延模型如下：
$$L = \frac{D}{0.6C}$$

其中， D 表示传播距离， C 表示光速，光纤中的传播速度近似取为 $0.6C$ 。

本实验的 Ping 测量值是往返时延 RTT，所以计算对应的 c-latency 时，需要用到往返距离： $c\text{-latency} = \frac{2D}{0.6C}$

数据处理所用的脚本 geo.py 如下：

```
1 import math
2 import pandas as pd
3 import geoip2.database
4
5 INPUT_FILE = "results.csv"
6 OUTPUT_FILE = "results_geo.csv"
7 DB_FILE = "dbip-city-lite-2026-09.mmdb"
8
9 CLIENT_LAT = 40.3160
10 CLIENT_LON = 116.6318
11
12
13 def haversine(lat1, lon1, lat2, lon2):
14     R = 6371.0
15
16     phi1 = math.radians(lat1)
17     phi2 = math.radians(lat2)
18
19     dphi = math.radians(lat2 - lat1)
20     dlamba = math.radians(lon2 - lon1)
21
22     a = (
23         math.sin(dphi / 2) ** 2
24         + math.cos(phi1)
25         * math.cos(phi2)
26         * math.sin(dlamba / 2) ** 2
27     )
28
29     c = 2 * math.atan2(
30         math.sqrt(a),
31         math.sqrt(1 - a)
32     )
33
34     return R * c
35
36
37 def c_latency_rtt_ms(distance_km):
38     speed_km_s = 0.6 * 300000
39     return (
40         2 * distance_km / speed_km_s
41     ) * 1000
42
43
44 df = pd.read_csv(INPUT_FILE)
45
46 reader = geoip2.database.Reader(DB_FILE)
47
48 server_latitudes = []
49 server_longitudes = []
```

```

50 countries = []
51 cities = []
52 distances = []
53 c_latencies = []
54
55 for ip in df["ip"]:
56
57     try:
58         response = reader.city(ip)
59
60         lat = response.location.latitude
61         lon = response.location.longitude
62
63         country = response.country.name
64         city = response.city.name
65
66         if lat is None or lon is None:
67             raise ValueError()
68
69         distance = haversine(
70             CLIENT_LAT,
71             CLIENT_LON,
72             lat,
73             lon
74         )
75
76         c_latency = c_latency_rtt_ms(distance)
77
78     except Exception:
79         lat = None
80         lon = None
81         country = None
82         city = None
83         distance = None
84         c_latency = None
85
86     server_latitudes.append(lat)
87     server_longitudes.append(lon)
88     countries.append(country)
89     cities.append(city)
90     distances.append(distance)
91     c_latencies.append(c_latency)
92
93 reader.close()
94
95 df["server_lat"] = server_latitudes
96 df["server_lon"] = server_longitudes
97 df["country"] = countries
98 df["city"] = cities
99 df["distance_km"] = distances
100 df["c_latency_ms"] = c_latencies
101
102 df.to_csv(

```

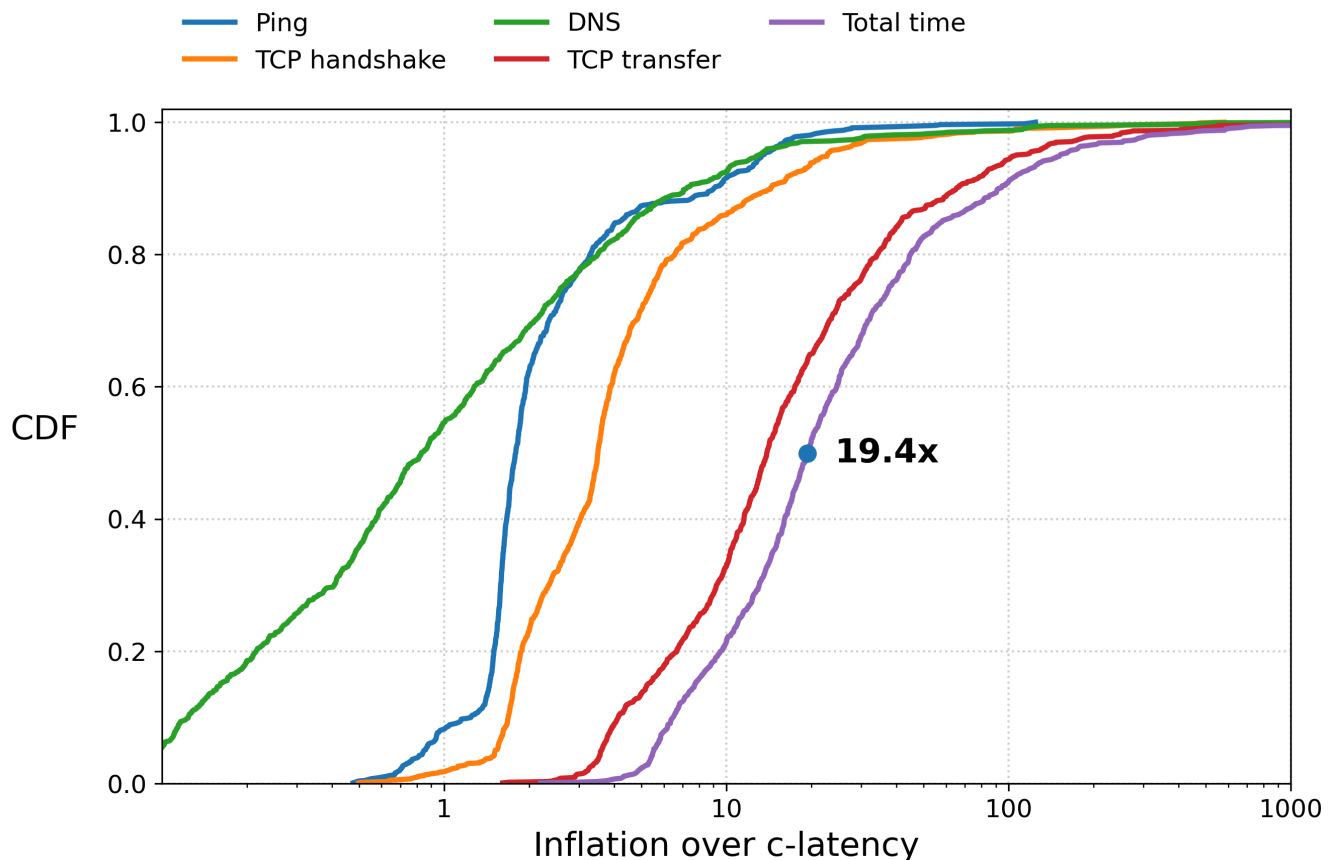
```

103     OUTPUT_FILE,
104     index=False
105 )
106
107 print(f"Saved to {OUTPUT_FILE}")

```

2.3 实验结果及分析

把各项实际测量时间除以 c-latency，得到相对理论传播时延的膨胀倍数，再据此绘制 CDF 曲线。



图中DNS和Ping的膨胀倍数整体较小，TCP handshake稍大一些，TCP transfer和Total time则明显更大；Total time的中位膨胀倍数约为19.4x，说明实际的完整传输时间通常比仅按传播距离计算的理论时延要长得多。

这一差异主要来自两方面，一是实际网络路径比地理直线距离更长，二是传输过程中还有发送延迟、队列延迟和TCP建连等额外开销。

部分样本的实际Ping小于估算的c-latency，主要是因为GeoIP只能大致估计服务器位置，加上CDN、云服务等机制的影响，实际响应节点可能与数据库记录的位置不同。

绘图使用的脚本 `plot.py` 如下：

```

1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  INPUT_FILE = "results_geo.csv"
6  OUTPUT_FILE = "cdf_inflation.png"
7
8  DRAW_ROUTER_PATH = False

```

```

9
10 def prepare_ratio(df, metric, c_col="c_latency_ms"):
11     data = df[[metric, c_col]].dropna().copy()
12
13     data = data[
14         (data[metric] > 0) &
15         (data[c_col] > 0)
16     ]
17
18     ratio = data[metric] / data[c_col]
19
20     ratio = ratio.replace([np.inf, -np.inf], np.nan).dropna()
21
22     return ratio.values
23
24
25 def ecdf(values):
26     x = np.sort(values)
27     y = np.arange(1, len(x) + 1) / len(x)
28     return x, y
29
30 def add_curve(ax, values, label, linewidth=2.5):
31     if len(values) == 0:
32         print(f"warning: no valid data for {label}")
33         return None
34
35     x, y = ecdf(values)
36
37     line, = ax.plot(
38         x,
39         y,
40         linewidth=linewidth,
41         label=label
42     )
43
44     return line
45
46
47 def main():
48     df = pd.read_csv(INPUT_FILE)
49
50     required = [
51         "ping_ms",
52         "dns_ms",
53         "tcp_handshake_ms",
54         "tcp_transfer_ms",
55         "total_ms",
56         "c_latency_ms",
57     ]
58
59     missing = [c for c in required if c not in df.columns]
60
61     if missing:

```

```

62         raise ValueError(
63             f"Missing columns in {INPUT_FILE}: {missing}"
64         )
65
66     ping_ratio = prepare_ratio(df, "ping_ms")
67     dns_ratio = prepare_ratio(df, "dns_ms")
68     handshake_ratio = prepare_ratio(df, "tcp_handshake_ms")
69     transfer_ratio = prepare_ratio(df, "tcp_transfer_ms")
70     total_ratio = prepare_ratio(df, "total_ms")
71
72     print("Valid samples:")
73     print("Min Ping:", len(ping_ratio))
74     print("DNS:", len(dns_ratio))
75     print("TCP handshake:", len(handshake_ratio))
76     print("TCP transfer:", len(transfer_ratio))
77     print("Total time:", len(total_ratio))
78
79     router_ratio = None
80
81     if DRAW_ROUTER_PATH:
82         if "router_path_ms" in df.columns:
83             router_ratio = prepare_ratio(df, "router_path_ms")
84         else:
85             print(
86                 "Warning: DRAW_ROUTER_PATH=True, "
87                 "but router_path_ms not found."
88             )
89
90     fig, ax = plt.subplots(figsize=(9, 6))
91
92     lines = []
93
94     if router_ratio is not None:
95         line = add_curve(
96             ax,
97             router_ratio,
98             "Router-path"
99         )
100         if line:
101             lines.append(line)
102
103     for values, label in [
104         (ping_ratio, "Ping"),
105         (handshake_ratio, "TCP handshake"),
106         (dns_ratio, "DNS"),
107         (transfer_ratio, "TCP transfer"),
108         (total_ratio, "Total time"),
109     ]:
110         line = add_curve(
111             ax,
112             values,
113             label
114         )

```

```

115
116         if line:
117             lines.append(line)
118
119     ax.set_xscale("log")
120
121     ax.set_xlim(0.1, 1000)
122     ax.set_ylim(0, 1.02)
123
124     ax.set_xlabel(
125         "Inflation over c-latency",
126         fontsize=16
127     )
128
129     ax.set_ylabel(
130         "CDF",
131         fontsize=16,
132         rotation=0,
133         labelpad=30
134     )
135
136     ax.grid(
137         True,
138         which="major",
139         linestyle=":",
140         linewidth=1,
141         alpha=0.6
142     )
143
144     ax.set_yticks(
145         np.arange(0, 1.01, 0.2)
146     )
147
148     ax.set_xticks(
149         [1, 10, 100, 1000]
150     )
151
152     ax.get_xaxis().set_major_formatter(
153         plt.ScalarFormatter()
154     )
155
156     ax.tick_params(
157         axis="both",
158         labelsize=12
159     )
160
161     ax.legend(
162         loc="lower left",
163         bbox_to_anchor=(0.0, 1.02),
164         ncol=3,
165         frameon=False,
166         fontsize=12
167     )

```

```

168
169     if len(total_ratio) > 0:
170         median_total = np.median(total_ratio)
171
172         x_total, y_total = ecdf(total_ratio)
173
174         idx = np.argmin(
175             np.abs(y_total - 0.5)
176         )
177
178         x50 = x_total[idx]
179         y50 = y_total[idx]
180
181         ax.scatter(
182             [x50],
183             [y50],
184             s=60,
185             zorder=5
186         )
187
188         ax.text(
189             x50 * 1.25,
190             y50,
191             f"{median_total:.1f}x",
192             fontsize=16,
193             fontweight="bold",
194             va="center"
195         )
196
197     plt.tight_layout()
198
199     plt.savefig(
200         OUTPUT_FILE,
201         dpi=300,
202         bbox_inches="tight"
203     )
204
205     print(f"Saved figure to {OUTPUT_FILE}")
206
207     plt.show()
208
209
210 if __name__ == "__main__":
211     main()

```

